

# Optimization Techniques

## Lab n°3 - Conjugate gradient descent algorithm

---

**Exercise 1** In this exercise, we will verify that the **conjugate gradient descent method**, applied to a quadratic function in  $n$  variables, converges (exactly) in at most  $n$  steps.

Write a program that works in **any** dimension  $n$ .

1. First, choose  $n$  (for instance  $n = 4, 5$ ) define a random quadratic function in  $n$  variables, as follows:

### Matlab Code

```
1: Q=rand(n,n);
2: Q=Q'+Q;
3: Q=Q+eye(n);
4: B=rand(1,n);
```

- ▷ define a random matrix
- ▷ make it symmetric, nonnegative
- ▷ shifts eigenvalues

### Python Code

```
1: import numpy as np
2: Q = np.random.rand(n, n)
3: Q = Q.T @ Q
4: Q = Q + np.eye(n)
5: B = np.random.rand(1, n)
```

- ▷ define a random matrix
- ▷ make it symmetric, nonnegative
- ▷ shift eigenvalues

2. Compute (and stock) the true minimum

$$\mathbf{x}_{th}^* = -A^{-1}B.$$

3. Choose random starting point  $\mathbf{x}_0$ .
4. Implement the conjugate gradient method, and verify that, after  $n$  steps, it converges to  $\mathbf{x}_{th}^*$ .

**Exercise 2** Modify the program of the previous exercise, introducing the (additional) stopping criterion  $\|\nabla f(\mathbf{x}_k)\| \leq 10^{-8}$ . Choose  $n \sim 500$ . After how many steps does the algorithm stop? Compute also  $\|\mathbf{x}_{th}^* - \mathbf{x}_k\|$ .

**Exercise 3** In this exercise we study the effect of preconditioning.

Save the file `Dubcova1.mtx`. It contains the entries of a sparse matrix of dimension  $16129 \times 16129$ . To read it, use the following commands:

### Matlab:

```
A = load('Dubcova1.mtx',' -ascii ');
A=spconvert(A);
Q=A+A' - diag(diag(Q));
n = size(Q, 1);
B = ones(n, 1);
```

⚠ spaces!

starting point

### Python:

```
from scipy.io import mmread
import numpy as np
A = mmread('Dubcova1.mtx')
Q=A+A.T - np.diag(np.diag(A))
n = Q.shape[0]
B = np.ones(n)
```

starting point

```

or
from scipy.io import mmread
from scipy.sparse import diags
A = mmread('Dubcova1.mtx')
Q=A+A.T - diags(A.diagonal())
n = Q.shape[0]
B = np.ones(n)

```

*starting point*

Then, implement the algorithm at page 25/28 of lecture slides.

For **Matlab** users: use the command `L = ichol(Q)`. Then, to obtain  $y_0$  such that  $My_0=r_0$  use forward and backward substitutions: `y0 = L' \ (L \ r0)`;

For **Python** users: the `scipy.sparse` library does not have a direct equivalent for `ichol`. Instead, you will use the Incomplete LU factorization, which returns a solver object rather than raw matrices. Import the function via from

```

scipy.sparse.linalg import spilu

```

and compute the preconditioner object with

```

ilu = spilu(Q, drop_tol=1e-4).

```

Then, to obtain  $y_0$  such that  $My_0=r_0$ , simply use the object's built-in method:

```

y0 = ilu.solve(r0)

```

Compare the number of iterations and the convergence time of the preconditioned and the non-preconditioned CGA.

**Exercise 4** If you still have time, compare the performances of the steepest gradient and the nonlinear conjugate gradient for the Rosenbrock's function

$$f(x,y) = 100(y - x^2)^2 + (1 - x)^2,$$

with starting point  $(-2, 2)$ .